



Vaam Ki Aawaz

Security Threat and Technical Bugs Analysis

Comprehensive Application Security & Code Quality Audit
Next.js CMS — Authentication, APIs, Content Publishing, and Engagement

Document date: 13 July 2026

Classification: Confidential — Internal Use

1. Executive Summary

This document records security vulnerabilities and technical defects identified during a full-codebase audit of the Vaam Ki Aawaz Next.js application (Prisma/MySQL, cookie JWT auth, rich-text publishing, OTP email flows, and public engagement APIs). Findings are based on code evidence. Product clarifications from stakeholders are incorporated: category mutation must require MASTER_ADMIN or manageCategories; publishers must not be able to execute scripts via article HTML while TipTap toolbar features remain; JWT_SECRET is set in Hostinger production; /api/debug must not be public; newsletter subscriptions must eventually persist securely in the database.

Overall Security Risk Rating: High | **Overall Code Quality Rating:** Fair — needs hardening and consistency fixes.

This report lists **22** security-related findings and **10** technical / reliability / maintainability findings.

2. Scope and Method

Review covered authentication and session handling, authorization on API routes, file uploads and media serving, OTP and mail flows, HTML rendering paths, engagement endpoints, configuration and debug surfaces, and selected reliability patterns (schema bootstrap, error handling, build config). Hypothetical issues without code evidence were excluded. Intended product behavior was confirmed with the project owner before final severity assignment for categories and rich-text XSS.

3. Ratings Legend

Critical — Direct, unauthenticated or trivial path to major integrity or takeover impact. **High** — Realistic exploit or serious abuse with significant impact. **Medium** — Meaningful risk or defect under common conditions. **Low** — Limited impact or harder prerequisites. **Informational** — Hygiene, incomplete feature, or defense-in-depth note.

4. Security Issues

This section covers authentication, authorization, injection/XSS, session handling, sensitive data exposure, upload risks, abuse of public APIs, and related cryptographic concerns.

4.1 Security Findings Summary

#	Severity	Title
1	Critical	Unauthenticated category create / hide
2	High	Hardcoded JWT secret fallback in source code
3	High	Public /api/debug information disclosure
4	High	Stored XSS via article HTML; ineffective sanitizers
5	High	OTP send/verify lacking rate limits and logout
6	High	JWT role trusted without database re-check
7	High	Login endpoint without rate limiting
8	High	File uploads allowed for any authenticated user
9	Medium	Open redirect via blog image API
10	Medium	Unauthenticated bootstrap setup endpoint and race
11	Medium	Unauthenticated comments without abuse controls
12	Medium	Click count and reaction inflation
13	Medium	Missing security headers and Content-Security-Policy
14	Medium	Resource detail endpoint returns full fileData payloads
15	Low	Inactive contributors can still obtain sessions
16	Low	OTP codes stored and compared in plaintext
17	Low	Contributor codes generated with Math.random
18	Low	API error responses leak internal exception messages
19	Low	Inline PDF media without Content-Disposition attachment
20	Low	No password strength policy on register/setup
21	Informational	CSRF residual risk with cookie sessions
22	Informational	Client-side OTP before user create not enforced server-side

4.2 Security Findings Detail

S-01. Unauthenticated category create / hide

Severity: Critical | **Category:** Broken Access Control | **Confidence:** High

Affected file(s): src/app/api/categories/route.ts; ClientPage.tsx (UI-only gate)

Evidence: POST /api/categories upserts categories with no requireAuth or permission check. Product intent confirmed: only MASTER_ADMIN or admins with manageCategories may change categories.

Why it is a problem: Authorization exists only in the UI. Attackers call the API directly and bypass the permission check.

Realistic impact: Any internet user can create fake categories or soft-hide real ones, disrupting navigation and publishing.

How to reproduce: POST /api/categories with JSON {"name":"x","isHidden":true} without authentication cookies.

Recommended fix: Require authentication; load user from DB; allow only MASTER_ADMIN or ADMIN with manageCategories === true; return 403 otherwise.

S-02. Hardcoded JWT secret fallback in source code

Severity: High | **Category:** Cryptographic / Authentication | **Confidence:** High

Affected file(s): src/lib/auth.ts (lines 5–6)

Evidence: Code uses `process.env.JWT_SECRET || "your-super-secret-jwt-key-replace-in-production"`. Production Hostinger env sets a long random `JWT_SECRET`, so the fallback is not active when the env var is present.

Why it is a problem: If the env var is ever missing on a deploy, the known default secret allows forging `MASTER_ADMIN` tokens.

Realistic impact: Misconfigured deploy could enable full administrative takeover of the CMS.

How to reproduce: Omit `JWT_SECRET` locally, mint a JWT with the fallback string and role `MASTER_ADMIN`, call privileged APIs.

Recommended fix: Fail application startup if `JWT_SECRET` is missing or weak; never ship a default secret. Rotate if any environment may have used the fallback.

S-03. Public /api/debug information disclosure

Severity: High | **Category:** Sensitive Information Disclosure | **Confidence:** High

Affected file(s): src/app/api/debug/route.ts

Evidence: Unauthenticated GET returns masked `DATABASE_URL` structure, DB connection test result, `NODE_ENV`, and `process.cwd()`. Confirmed: must not remain public in production.

Why it is a problem: Exposes infrastructure and environment details to anyone who can reach the endpoint.

Realistic impact: Aids reconnaissance (DB host/user shape, deploy paths, environment). Connection failures may leak further detail.

How to reproduce: GET `/api/debug` without authentication.

Recommended fix: Remove the route in production, or restrict to `MASTER_ADMIN` and non-production environments only.

S-04. Stored XSS via article HTML; ineffective sanitizers

Severity: High | **Category:** Cross-Site Scripting (XSS) | **Confidence:** High

Affected file(s): src/utls/sanitizeHtml.tsx; src/utls/designUtils.ts (cleanHtml); ArticlePage.tsx; TiptapEditor GenericIframe / Link

Evidence: `sanitizeHtmlClient` only normalizes whitespace; `cleanHtml` only strips `ZWSP/nbsp`. Content is rendered with `dangerouslySetInnerHTML`. Product intent: publishers must not run scripts; TipTap toolbar features (links, images, video embeds, formatting) must remain.

Why it is a problem: Dangerous tags and event handlers in published HTML execute for every reader. Same-origin XSS can drive authenticated admin actions while an admin views a post.

Realistic impact: Compromised or malicious publisher account can attack readers and logged-in staff. Direct API posts can bypass any client-only checks.

How to reproduce: Publish article HTML containing e.g. `<img src=x onerror=alert(1)>` or a script tag via editor paste or API.

Recommended fix: Server-side HTML allowlist on blog create/update (and ideally on render) that permits TipTap-safe tags/attrs and YouTube/Vimeo iframes only; strip script, on*, javascript: URLs. Do not change TipTap toolbar UX. Add CSP as defense in depth.

S-05. OTP send/verify lacking rate limits and logout

Severity: High | **Category:** API Abuse / Weak OTP | **Confidence:** High

Affected file(s): src/app/api/otp/send/route.ts; src/app/api/otp/verify/route.ts

Evidence: Any email can request OTPs with no throttle; 6-digit codes; no verify attempt limit; codes stored and compared in plaintext.

Why it is a problem: Enables SMTP/email bombing and OTP guessing for flows that trust verification.

Realistic impact: Inbox flooding via site SMTP credentials; abuse of newsletter/admin UX OTP steps; resource cost on mail provider.

How to reproduce: Repeatedly POST `/api/otp/send` for a victim email; brute-force `/api/otp/verify`.

Recommended fix: Per-IP and per-email rate limits; exponential backoff; hash OTPs at rest; max attempts; bind OTP to purpose/session.

S-06. JWT role trusted without database re-check

Severity: High | **Category:** Authorization / Session Management | **Confidence:** High

Affected file(s): src/app/api/events/*; src/app/api/resources/*; src/app/reports/page.tsx

Evidence: These paths check authPayload.role / session.role from the JWT instead of reloading current role from the database (unlike blogs/users/site-config).

Why it is a problem: Role demotion does not revoke elevated access until the 24h token expires.

Realistic impact: A demoted former MASTER_ADMIN can still manage events, resources, and reports for up to one day.

How to reproduce: Log in as MASTER_ADMIN, demote the account in DB, continue calling event/resource APIs with the same cookie.

Recommended fix: Always authorize from the current DB user record; invalidate or shorten sessions on role/active changes.

S-07. Login endpoint without rate limiting

Severity: High | **Category:** Authentication Abuse | **Confidence:** High

Affected file(s): src/app/api/auth/login/route.ts

Evidence: Unlimited password attempts; each verifyPassword uses scrypt (CPU-costly).

Why it is a problem: Enables credential stuffing and login-endpoint CPU exhaustion.

Realistic impact: Account compromise risk and potential DoS against the auth path.

How to reproduce: Script repeated POST /api/auth/login with varying passwords.

Recommended fix: Rate limit by IP and email; lockout or CAPTCHA after N failures.

S-08. File uploads allowed for any authenticated user

Severity: High | **Category:** Broken Access Control / File Upload | **Confidence:** High

Affected file(s): src/app/api/uploads/route.ts; src/lib/fileStorage.ts

Evidence: Only requireAuth is checked; inactive contributors can upload; MIME type taken from client file.type without magic-byte verification.

Why it is a problem: Upload privilege is broader than publishing privilege; client-claimed MIME is spoofable.

Realistic impact: Disk abuse by deactivated accounts; risk of storing non-image content under image extensions.

How to reproduce: Authenticate as inactive contributor; POST multipart file to /api/uploads.

Recommended fix: Require publish/admin capability and active status; verify file signatures; enforce quotas.

S-09. Open redirect via blog image API

Severity: Medium | **Category:** Open Redirect | **Confidence:** High

Affected file(s): src/app/api/image/blog/{id}/route.ts

Evidence: Redirects to http(s) URLs or resolves relative/protocol-relative URLs from postImage or first in content.

Why it is a problem: Trusted site URLs can bounce users to attacker-controlled destinations.

Realistic impact: Phishing and trust abuse via shared /api/image/blog/{id} links.

How to reproduce: Publish content with external or //evil.example image src; visit /api/image/blog/{id}.

Recommended fix: Serve only local media or data:image URIs; never redirect to arbitrary hosts.

S-10. Unauthenticated bootstrap setup endpoint and race

Severity: Medium | **Category:** Authentication | **Confidence:** High

Affected file(s): src/app/api/auth/setup/route.ts

Evidence: POST is public until a MASTER_ADMIN exists; count-then-create is not transactional.

Why it is a problem: On an empty database, anyone can claim the first master admin; concurrent requests may race.

Realistic impact: Hostile first-writer takeover during initial setup or if the user table is wiped.

How to reproduce: On empty DB, POST /api/auth/setup with email/password; optionally race two requests.

Recommended fix: Gate with a setup secret or one-time env flag; enforce uniqueness/transactional create.

S-11. Unauthenticated comments without abuse controls

Severity: Medium | **Category:** API Abuse | **Confidence:** High

Affected file(s): src/app/api/blogs/[id]/comments/route.ts

Evidence: POST accepts name/email/comment with no auth, OTP, CAPTCHA, or rate limit; stores email.

Why it is a problem: Public write endpoint is trivial to automate.

Realistic impact: Spam, PII pollution, and database growth.

How to reproduce: Loop POST /api/blogs/{id}/comments with arbitrary payloads.

Recommended fix: Rate limit, CAPTCHA or verified identity, moderation queue.

S-12. Click count and reaction inflation

Severity: Medium | **Category:** Integrity / Abuse | **Confidence:** High

Affected file(s): src/app/api/blogs/[id]/click/route.ts; reactions/route.ts

Evidence: Unauthenticated click increment; visitorId fully client-controlled for reactions.

Why it is a problem: Popularity metrics are not integrity-protected.

Realistic impact: Fake rankings and misleading engagement statistics.

How to reproduce: Script POST click/reaction with rotating visitorIds.

Recommended fix: Rate limits, stronger visitor binding, anomaly detection.

S-13. Missing security headers and Content-Security-Policy

Severity: Medium | **Category:** Missing Security Headers | **Confidence:** High

Affected file(s): next.config.ts

Evidence: No CSP, X-Frame-Options, Referrer-Policy, or related hardening headers configured.

Why it is a problem: Browser-side defenses against XSS and clickjacking are weaker than they should be.

Realistic impact: Higher impact if HTML injection occurs; easier framing attacks.

How to reproduce: Inspect response headers on production pages.

Recommended fix: Add CSP (compatible with TipTap/embeds), frame protections, and standard security headers in Next config.

S-14. Resource detail endpoint returns full fileData payloads

Severity: Medium | **Category:** Data Exposure / DoS | **Confidence:** High

Affected file(s): src/app/api/resources/[id]/route.ts

Evidence: GET returns the full resource including large base64 fileData when present.

Why it is a problem: Large binary blobs should be served as media URLs, not embedded in JSON.

Realistic impact: Bandwidth/memory pressure; unnecessary exposure of full file contents in API responses.

How to reproduce: GET /api/resources/{id} for a PDF stored as data URI/fileData.

Recommended fix: Store on disk/media only; return URL references, not raw base64 in list/detail JSON.

S-15. Inactive contributors can still obtain sessions

Severity: Low | **Category:** Session / Authorization | **Confidence:** High

Affected file(s): src/app/api/auth/login/route.ts

Evidence: Login does not reject active === false; cookie is still issued (publish routes correctly block inactive publishers).

Why it is a problem: Deactivation does not fully revoke interactive access.

Realistic impact: Combined with unrestricted uploads, deactivated users retain unnecessary capabilities.

How to reproduce: Deactivate a contributor; log in successfully; call /api/uploads.

Recommended fix: Reject login for inactive contributors; optionally revoke outstanding sessions.

S-16. OTP codes stored and compared in plaintext

Severity: Low | **Category:** Cryptography | **Confidence:** High

Affected file(s): OTP send/verify routes; Otp model

Evidence: Codes inserted and selected as plaintext strings with equality compare.

Why it is a problem: DB disclosure of OTP table yields usable codes within expiry.

Realistic impact: Elevates impact of any database read compromise affecting OTP rows.

How to reproduce: N/A (inspect schema/code path).

Recommended fix: Store salted hashes; use constant-time compare; short TTL and single use.

S-17. Contributor codes generated with Math.random

Severity: Low | **Category:** Cryptography | **Confidence:** High

Affected file(s): src/lib/contributorCode.ts

Evidence: Suffix characters chosen via Math.random rather than crypto.randomInt.

Why it is a problem: Non-cryptographic RNG is predictable relative to CSPRNG.

Realistic impact: Lower practical risk for display codes, but poor crypto hygiene.

How to reproduce: N/A.

Recommended fix: Use crypto.randomInt / randomBytes.

S-18. API error responses leak internal exception messages

Severity: Low | **Category:** Information Disclosure | **Confidence:** High

Affected file(s): Multiple API routes (blogs, OTP, authors, resources, etc.)

Evidence: Responses include details: err.message or similar internal strings.

Why it is a problem: Helps attackers map stack, schema, and misconfigurations.

Realistic impact: Reconnaissance aid; usually not direct RCE.

How to reproduce: Trigger failures (bad schema, invalid IDs) and inspect JSON error bodies.

Recommended fix: Return generic client errors; log details server-side only.

S-19. Inline PDF media without Content-Disposition attachment

Severity: Low | **Category:** Content Handling | **Confidence:** Medium

Affected file(s): src/app/api/media/[...path]/route.ts

Evidence: PDFs served with Content-Type application/pdf and no attachment disposition; no nosniff header.

Why it is a problem: Browser PDF viewers have historically had XSS-class issues with crafted PDFs.

Realistic impact: Browser-dependent risk for malicious PDFs uploaded by privileged users.

How to reproduce: Upload PDF and open /api/media/resources/... in browser.

Recommended fix: Add Content-Disposition: attachment for PDFs and X-Content-Type-Options: nosniff.

S-20. No password strength policy on register/setup

Severity: Low | **Category:** Password Handling | **Confidence:** High

Affected file(s): src/app/api/auth/register/route.ts; setup/route.ts

Evidence: Only presence of password is checked; hashing via bcrypt is otherwise sound.

Why it is a problem: Weak passwords increase credential stuffing success.

Realistic impact: Easier compromise of staff accounts if weak passwords are chosen.

How to reproduce: Create user with a short password.

Recommended fix: Enforce minimum length and basic complexity server-side.

S-21. CSRF residual risk with cookie sessions

Severity: Informational | **Category:** CSRF | **Confidence:** Medium

Affected file(s): Login/logout cookie settings (sameSite: lax, httpOnly)

Evidence: Auth uses httpOnly cookie with SameSite=Lax; no CSRF tokens on state-changing APIs.

Why it is a problem: Lax mitigates many classic cross-site POSTs; XSS remains the more realistic abuse path.

Realistic impact: Limited for classic CSRF; still worth defense-in-depth for sensitive actions.

How to reproduce: N/A.

Recommended fix: Consider CSRF tokens or SameSite=Strict for admin mutations; prioritize XSS fixes.

S-22. Client-side OTP before user create not enforced server-side

Severity: Informational | **Category:** Authorization / Business Logic | **Confidence:** High

Affected file(s): ClientPage OTP flow; src/app/api/auth/register/route.ts

Evidence: OTP verify is a UI gate; /api/auth/register does not require a verified OTP token.

Why it is a problem: Email verification of new admins/contributors can be skipped via direct API use by an authorized admin.

Realistic impact: Weaker assurance that the email mailbox is controlled by the new user.

How to reproduce: As MASTER_ADMIN, POST /api/auth/register without completing OTP UI.

Recommended fix: Issue a short-lived server-side verification token after OTP and require it on register.

5. Technical Bugs and Reliability Issues

This section covers logic bugs, incomplete features, concurrency and performance problems, build/configuration reliability issues, and maintainability concerns that affect correctness or operational safety even when they are not classic external exploit classes.

5.1 Technical Findings Summary

#	Severity	Title
1	Medium	ADMIN manageUsers permission not parsed on list/register APIs
2	Medium	Newsletter form does not persist subscriptions
3	Medium	Runtime DDL on hot API request paths
4	Medium	Unbounded article content and data-URI images in database
5	Medium	TypeScript build errors ignored in Next config
6	Low	ensureBlogSchema / seed race on empty blog table
7	Low	Monolithic ClientPage maintainability risk
8	Low	Inconsistent authorization patterns across APIs
9	Informational	jose package not declared as direct dependency
10	Informational	Misleading HTML sanitizer naming and behavior

5.2 Technical Findings Detail

T-01. ADMIN manageUsers permission not parsed on list/register APIs

Severity: Medium | **Category:** Logic / Authorization Bug | **Confidence:** High

Affected file(s): src/app/api/users/route.ts; src/app/api/auth/register/route.ts (users/[id] parses correctly)

Evidence: permissions is stored as a JSON string but cast directly to an object; .manageUsers is never true for ADMINS.

Why it is a problem: Delegated user-management permission is broken on the server while the UI shows it as enabled.

Realistic impact: Only MASTER_ADMIN can list/create users via API; admins with manageUsers see UI that fails silently.

How to reproduce: Grant manageUsers to an ADMIN; call GET /api/users — expect 403.

Recommended fix: Use parseUserPermissions consistently on all authz checks.

T-02. Newsletter form does not persist subscriptions

Severity: Medium | **Category:** Incomplete Feature / Data Integrity | **Confidence:** High

Affected file(s): ClientPage.tsx (executeNewsletterSubscription); OTP routes

Evidence: After OTP success, the client only shows a thank-you message and clears fields. No newsletter table/API write. Product intent: store subscriber details securely in the database when activated.

Why it is a problem: Users believe they subscribed; data is discarded.

Realistic impact: Lost subscribers; false UX; OTP cost without business value.

How to reproduce: Complete newsletter OTP flow; verify no DB row is created.

Recommended fix: Add NewsletterSubscriber model/API with validation, rate limits, consent fields, and secure storage; wire OTP verify to a server-side subscribe action.

T-03. Runtime DDL on hot API request paths

Severity: Medium | **Category:** Reliability / Performance | **Confidence:** High

Affected file(s): src/lib/db-setup.ts; OTP/events/authors routes using CREATE TABLE / ALTER

Evidence: Schema ensure functions run during request handling.

Why it is a problem: DDL under load causes locks, latency spikes, and racey schema evolution.

Realistic impact: Intermittent slow requests and production risk during traffic peaks.

How to reproduce: Hit /api/blogs or OTP under concurrency while schema ensure runs.

Recommended fix: Use Prisma migrations at deploy time only; remove request-path DDL.

T-04. Unbounded article content and data-URI images in database

Severity: Medium | **Category:** Performance / DoS | **Confidence:** High

Affected file(s): src/app/api/blogs/route.ts; src/lib/fileStorage.ts isValidImageRef

Evidence: No max content length; data:image references are accepted as valid image refs.

Why it is a problem: Huge LongText payloads inflate DB and memory for list/detail operations.

Realistic impact: Slow pages, high memory use, possible request timeouts.

How to reproduce: POST a multi-megabyte HTML body or large data URI as postImage.

Recommended fix: Enforce body size limits; require disk/media URLs for images.

T-05. TypeScript build errors ignored in Next config

Severity: Medium | **Category:** Reliability | **Confidence:** High

Affected file(s): next.config.ts

Evidence: typescript.ignoreBuildErrors: true

Why it is a problem: Type errors do not fail CI/production builds.

Realistic impact: Runtime exceptions from preventable type bugs ship to production.

How to reproduce: Introduce a type error; next build still succeeds.

Recommended fix: Remove the flag and fix type errors.

T-06. ensureBlogSchema / seed race on empty blog table

Severity: Low | **Category:** Concurrency / Logic | **Confidence:** Medium

Affected file(s): src/app/api/blogs/route.ts GET

Evidence: count === 0 then createMany of seed posts without strong concurrency control.

Why it is a problem: Parallel cold starts can duplicate seed attempts (createMany may partially conflict).

Realistic impact: Noisy errors or duplicate seed attempts on first deploy.

How to reproduce: Concurrent GET /api/blogs against empty BlogPost table.

Recommended fix: Seed in migration/deploy script; or use a transactional advisory lock.

T-07. Monolithic ClientPage maintainability risk

Severity: Low | **Category:** Maintainability | **Confidence:** High

Affected file(s): src/app/ClientPage.tsx (~4300+ lines)

Evidence: Homepage, admin, newsletter, resources, events, and auth UX colocated in one client component.

Why it is a problem: Hard to review, test, and securely change authorization-sensitive UI.

Realistic impact: Higher chance of regressions and missed authz gaps in future changes.

How to reproduce: N/A (structural).

Recommended fix: Split by feature routes/components without changing behavior when refactoring is explicitly requested.

T-08. Inconsistent authorization patterns across APIs

Severity: Low | **Category:** Architecture / Maintainability | **Confidence:** High
Affected file(s): Events/resources (JWT role) vs blogs/users/site-config (DB user)
Evidence: Two different authz styles coexist.
Why it is a problem: Easy to introduce privilege bugs when adding endpoints.
Realistic impact: Ongoing security debt and review cost.
How to reproduce: Compare requireAuth usage across api folder.
Recommended fix: Central helper: requireUser() returning DB role + parsed permissions.

T-09. jose package not declared as direct dependency

Severity: Informational | **Category:** Dependency Hygiene | **Confidence:** High
Affected file(s): src/lib/auth.ts; package.json
Evidence: jose is imported for JWT but only appears transitively in the lockfile.
Why it is a problem: Upstream dependency changes can break auth builds unexpectedly.
Realistic impact: Fragile installs / future breakage.
How to reproduce: Inspect package.json vs import of jose.
Recommended fix: Add jose as a direct dependency with a pinned version.

T-10. Misleading HTML sanitizer naming and behavior

Severity: Informational | **Category:** Code Smell / Security Debt | **Confidence:** High
Affected file(s): src/utils/sanitizeHtml.tsx; designUtils cleanHtml
Evidence: Names imply XSS protection; implementations only normalize whitespace.
Why it is a problem: Developers may assume content is safe when it is not.
Realistic impact: False sense of security; delays real XSS remediation.
How to reproduce: Read sanitizeHtmlClient implementation.
Recommended fix: Rename or replace with a real allowlist sanitizer; document TipTap-safe policy.

6. Most Urgent Fixes (Top 10)

- 1 Authenticate and authorize POST /api/categories (MASTER_ADMIN or manageCategories).
- 2 Remove or lock down /api/debug in production.
- 3 Fail closed if JWT_SECRET is missing; keep production secret strong and unique.
- 4 Rate-limit login and OTP send/verify; hash OTPs.
- 5 Server-side TipTap-safe HTML allowlist on blog create/update (no scripts; keep toolbar features).
- 6 Re-check role/permissions from the database for events, resources, and reports.
- 7 Restrict uploads by role/active status; verify file magic bytes.
- 8 Fix parseUserPermissions on users list and register routes.
- 9 Close open redirect in /api/image/blog/{id}.
- 10 Stop runtime DDL; use migrations; remove ignoreBuildErrors; implement secure newsletter persistence.

7. Potential Attack Scenarios

- 1 Anonymous category defacement via unauthenticated POST /api/categories.
- 2 Stored XSS from publisher HTML leading to reader compromise or admin session action abuse.
- 3 OTP/SMTP abuse by flooding /api/otp/send against arbitrary addresses.
- 4 Privilege persistence: demoted MASTER_ADMIN continues using JWT for events/resources/reports.
- 5 Open-redirect phishing via /api/image/blog/{id} after planting external image URLs in content.
- 6 Credential stuffing against /api/auth/login without rate limits.

8. Technical Debt Summary

The codebase mixes strong patterns (bcrypt password hashing, httpOnly cookies, many routes that re-load users from the database) with weak ones (JWT-role checks, public mutating endpoints, runtime schema DDL, and a very large ClientPage). Misleading sanitizer helpers create a false sense of XSS safety. Engagement and OTP endpoints lack abuse controls. Completing newsletter persistence and unifying authorization helpers would reduce both security and operational risk.

9. Recommended Next Steps

- 1 Patch Critical and High security items first (categories, debug, XSS sanitization, rate limits, JWT-role checks, uploads).
- 2 Add automated authorization tests for every mutating API route.
- 3 Implement TipTap-compatible server-side HTML sanitization without changing the editor toolbar.
- 4 Design and ship a secure NewsletterSubscriber API before activating the public form.
- 5 Run dependency vulnerability scanning (npm audit / Dependabot) on a recurring schedule.
- 6 Re-test after remediation and update this document's status column in a follow-up revision.

— End of Report —